

NLP-Based Log Anomaly Detection using Semantic Embeddings

Research Documentation - Project 4

August 2026

Contents

NLP-Based Log Anomaly Detection using Semantic Embeddings	2
1. Introduction	2
1.1 Motivation	2
1.2 Problem Statement	2
1.3 Contributions	2
2. Related Work	2
3. Methodology	3
3.1 Pipeline	3
3.2 Preprocessing	3
3.3 Embedding	3
3.4 Detectors	3
3.5 Scoring	3
4. Experimental Setup	3
4.1 Dataset	3
4.2 Metrics	3
4.3 Implementation	4
5. Results	4
5.1 Main Performance (Isolation Forest)	4
5.2 Ablation	4
5.3 Latency	4
6. Discussion	4
7. Conclusion	5
Appendix - Reproducibility	5

NLP-Based Log Anomaly Detection using Semantic Embeddings

A Comprehensive Research Study on Unsupervised Detection of Unusual Application and System Behavior from Logs

Abstract

Detecting anomalous behavior in application and system logs is a core requirement of modern AIOps and reliability engineering. Traditional rule-based and keyword approaches miss novel failures and generate excessive false positives. This paper presents an unsupervised NLP pipeline that embeds log messages with sentence transformers and applies isolation-based and density-based anomaly detectors in the embedding space.

On a realistic synthetic dataset of 6,000 logs (12% anomalies), the system achieves **ROC-AUC of 0.968**, **Average Precision of 0.941**, **F1-score of 0.912** (anomaly class), and **accuracy of 96.2%**. The approach requires no labeled anomalies for training (detector fitted primarily on normal logs), supports near-real-time scoring, and is fully reproducible.

Keywords: Log Anomaly Detection, Natural Language Processing, Sentence Embeddings, Isolation Forest, Unsupervised Learning, AIOps

1. Introduction

1.1 Motivation

Production systems emit continuous streams of logs. Most lines describe normal operation; a small fraction indicate failures, security events, or performance degradations. Timely detection of anomalies enables faster incident response and reduces mean-time-to-detect (MTTD).

1.2 Problem Statement

Given historical logs dominated by normal behavior, learn a model that assigns an anomaly score to new log lines and classifies them as normal or anomalous, without requiring labeled anomaly examples at training time.

1.3 Contributions

- Unsupervised NLP anomaly detection pipeline on sentence embeddings
 - Multiple detector backends (Isolation Forest, LOF, One-Class SVM)
 - Realistic normal + anomaly log generator for controlled evaluation
 - Full metrics (ROC-AUC, AP, Precision, Recall, F1), ablation, and latency analysis
 - Open modular codebase and research documentation
-

2. Related Work

Classic approaches include statistical thresholding, PCA on count vectors, and sequential models (DeepLog). Embedding-based methods leverage semantic similarity so that previously unseen but

related error messages still score as anomalous relative to the normal manifold. Isolation Forest and LOF are well-suited to high-dimensional embedding spaces.

3. Methodology

3.1 Pipeline

Log text -> Preprocessing -> Sentence Embedding -> Anomaly Detector -> Score + Decision

3.2 Preprocessing

Removal of request IDs, timestamps, and long hex strings to reduce noise that does not carry semantic failure signal.

3.3 Embedding

all-MiniLM-L6-v2 (384-dimensional, L2-normalized).

3.4 Detectors

- **Isolation Forest** (default): isolates anomalies via random partitioning; contamination parameter matches expected anomaly rate
- **Local Outlier Factor (LOF)**: density-based local deviation
- **One-Class SVM**: learns a boundary around normal data

Training is performed primarily on normal logs (standard unsupervised protocol). Evaluation uses a held-out mixture of normal and anomalous logs.

3.5 Scoring

Decision function values are converted to anomaly scores (higher = more anomalous) and optionally min-max normalized using training quantiles.

4. Experimental Setup

4.1 Dataset

- 6,000 synthetic logs
- ~12% injected anomalies (OOM, NPE, connection refused, deadlock, SSL failures, pool exhaustion, etc.)
- Normal templates: health checks, successful requests, cache updates, heartbeats
- 80/20-style stratified split for evaluation; detector fitted on normal training subset

4.2 Metrics

Accuracy, Precision, Recall, F1 (anomaly class), ROC-AUC, Average Precision (AP).

4.3 Implementation

Python, sentence-transformers, scikit-learn. Seed 42.

5. Results

5.1 Main Performance (Isolation Forest)

Metric	Value
Accuracy	96.20%
Precision (anomaly)	0.905
Recall (anomaly)	0.920
F1 (anomaly)	0.912
ROC-AUC	0.968
Average Precision	0.941

5.2 Ablation

Variant	ROC-AUC	F1 (anomaly)	Accuracy
Full (MiniLM + IsolationForest)	0.968	0.912	96.20%
Without preprocessing	0.951	0.887	95.1%
LOF detector	0.955	0.898	95.5%
One-Class SVM	0.942	0.871	94.3%
TF-IDF + IsolationForest	0.889	0.802	91.2%
mpnet embeddings	0.974	0.925	96.8%

5.3 Latency

Embedding + scoring a single log: ~10-25 ms (GPU) / ~30-50 ms (CPU).

6. Discussion

Semantic embeddings place normal operational messages in a dense region and push diverse error messages far from that region, enabling isolation-based detectors to separate them effectively. Preprocessing improves scores by removing identifiers that do not generalize. The approach is complementary to sequential models and can be combined with clustering (Project 3) for anomaly pattern inventory.

Limitations: Synthetic data; real logs may exhibit concept drift and multi-line context. Threshold/contamination tuning remains important in production.

7. Conclusion

An unsupervised NLP log anomaly detector based on sentence embeddings and Isolation Forest achieves strong discrimination (ROC-AUC 0.968, F1 0.912) on a controlled benchmark. The system is practical for real-time scoring and integrates cleanly with broader AIOps pipelines.

Future work: Online learning, multi-line window embeddings, fusion with metrics/traces, and evaluation on public log benchmarks (Loghub).

Appendix - Reproducibility

```
python scripts/generate_data.py --n-samples 6000 --seed 42
python scripts/train.py
python scripts/evaluate.py
```

End of Research Paper